24
랭체인으로 만드는 RAG

LLM이 모르는 내 문서를 찾아서 프롬프트에 덧붙여 주고 답하게 하는 기술입니다. 임베딩, 청킹, 벡터 데이터베이스, 프롬프트를 하나씩 만들어 본 뒤, 한국은행 용어집으로 답하는 금융 챗봇으로 묶습니다.

오늘 다루는 것

1. 랭체인을 시작하기 전에
2. RAG 이해하기: 임베딩과 코사인 유사도
3. 랭체인의 두 가지 청킹 전략
4. ChatOpenAI와 Memory
5. 랭체인의 다양한 PDF 로더
6. 벡터 데이터베이스
7. RAG를 이용한 한국은행 금융 챗봇

끝나고 나면 할 수 있는 것

1. RAG의 두 단계(미리 색인하기, 질문이 올 때 찾아서 덧붙여 쓰기)를 그림 없이 설명할 수 있다.
2. 문서를 청크로 자르고, 임베딩해서 벡터 데이터베이스에 넣고, 질문과 가까운 청크를 찾아올 수 있다.
3. 검색 결과를 프롬프트에 넣어 LLM이 근거를 가지고 답하게 하는 챗봇을 만들 수 있다.

2026년 10월 16일 금요일 14:30 (10/16 자료 4/5)

교재는 OpenAI API(유료 키)를 씁니다. 이 자료의 실습은 키 없이 돌아가도록 임베딩과 LLM을 내 컴퓨터에서 도는 공개 모델로 바꿨습니다. 처음 실행할 때 모델을 내려받습니다(임베딩 440MB, LLM 3GB).

24

랭체인으로 만드는 RAG

21장의 GPT는 학습한 글에 나온 것만 압니다. 우리 회사 규정집이나 어제 나온 보고서는 모릅니다. 모르는 것을 물으면 그럴듯하게 지어내기도 합니다. RAG(Retrieval-Augmented Generation, 검색 증강 생성)는 질문과 관련된 문서를 먼저 찾아서(검색), 프롬프트에 덧붙여(증강), 그것을 보고 답하게(생성) 하는 방법입니다. 모델을 다시 학습시키지 않습니다.

왜 배우나: 오늘의 학습 경로



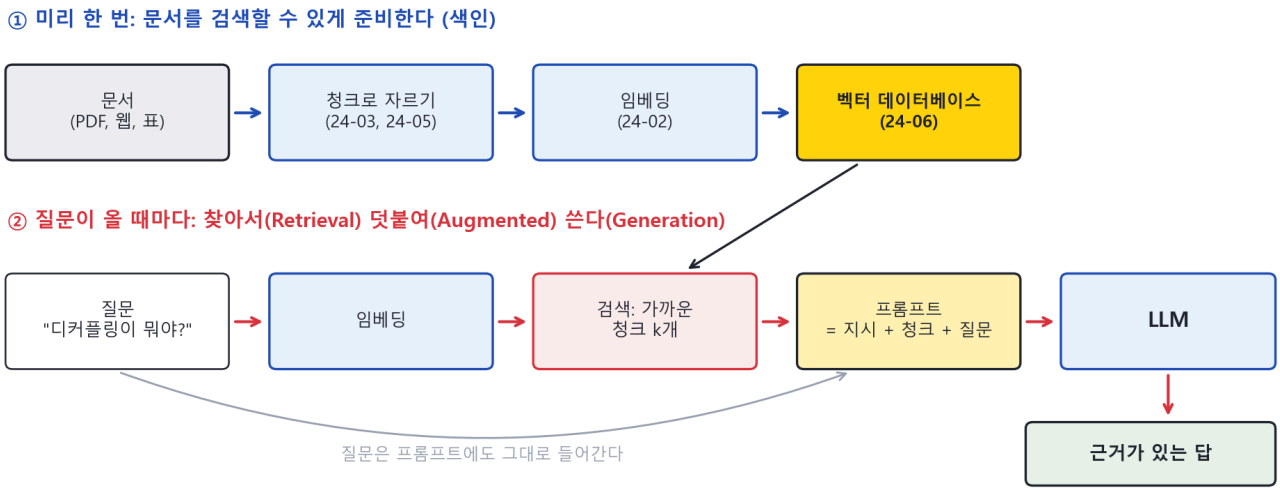


그림 1 RAG의 전체 흐름. 위는 미리 한 번 하는 일, 아래는 질문이 올 때마다 하는 일. 괄호는 그 부품을 다루는 절

20-06의 검색 기반 챗봇과 20-07의 시맨틱 검색기가 이 그림의 "검색"에 해당합니다. 거기에 LLM을 붙이면 RAG입니다. 예시는 교재를 따라 한국은행의 「경제금융용어 700선」으로 답하는 챗봇으로 갑니다.

24-01 랭체인을 시작하기 전에

OpenAI API 대신 로컬 모델을 썼습니다

역할	교재	이 자료
임베딩	OpenAIEmbeddings (text-embedding-3-small)	HuggingFaceEmbeddings (sentence-transformers/all-MiniLM-L6-v2)
LLM	ChatOpenAI (gpt-4o)	Qwen2.5-1.5B-Instruct (파라미터 15억 개의 공개 모델)

핵심 한 줄 랭체인은 LLM 서비스에 필요한 부품(문서 로더, 분할기, 임베딩, 벡터 데이터베이스, 프롬프트, LLM)을 같은 문법으로 묶어 주는 도구일 뿐이고, 배워야 할 것은 도구가 아니라 그 부품들이 하는 일이다.

왜 배우나 RAG를 만들 때 가장 널리 쓰이는 도구가 랭체인입니다. 이 장의 코드에 계속 나오므로 무엇을 해 주는 도구인지 먼저 알아 둡니다. 동시에, 이 도구 없이도 같은 것을 만들 수 있어야 한다는 교재의 당부도 같이 봅니다.

코드의 흐름과 랭체인 사용법은 교재와 같습니다. 다만 LLM이 gpt-4o보다 훨씬 작아 답의 품질은 떨어집니다. API 키가 있다면 교재의 `ChatOpenAI`, `OpenAIEmbeddings`로 바꿔 끼우면 됩니다. 바꿔 끼울 자리는 각 실습에 표시해 두었습니다.

1) 랭체인이 해 주는 일

정의

- 랭체인(LangChain): LLM을 이용한 서비스를 만들 때 쓰는 파이썬 프레임워크. 여러 회사의 LLM, 여러 형식의 문서, 여러 벡터 데이터베이스를 같은 방식의 코드로 다루게 해 줍니다.
- 프레임워크(framework): 자주 쓰는 기능을 미리 만들어 두고 정해진 방식대로 조립해 쓰게 한 도구 모음.

장점	내용
데이터 소스의 통합	PDF, 웹페이지, CSV, 데이터베이스를 읽는 코드가 같은 모양이다
개발 생산성	대화 기록 관리, 프롬프트 조립, 부품 연결을 직접 짜지 않아도 된다

주의. 랭체인은 필수가 아닙니다. 지나친 추상화가 원리를 가리기도 합니다. 교재는 "랭체인을 빼면 RAG를 구현하지 못하는" 경우를 경계하며, 도구가 아니라 기술의 본질을 공부하라고 당부합니다.

예시 적용 이 자료는 그 당부를 따라, 각 부품을 먼저 랭체인 없이(또는 최소한으로) 만들어 본 뒤 랭체인의 같은 부품을 씁니다.

부품	직접 만들어 본 곳	랭체인의 부품
검색	실습 1: 넘파이와 판다스로 코사인 유사도 검색	Chroma, FAISS (실습 5)
의미 고려 청킹	실습 2: 이웃 문장의 거리로 직접 자르기	SemanticChunker
RAG	실습 6: 검색 → 문자열 합치기 → LLM 호출을 함수 하나로	리트리버, 프롬프트 템플릿

24-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 랭체인이 없으면 RAG를 구현할 수 없다. (O / X) 이름 _____

2 랭체인은 그 자체가 하나의 LLM이다. (O / X) 이름 _____

3 RAG에서 R이 뜻하는 것은? 이름 _____

① 검색(Retrieval) ② 추론(Reasoning) ③ 순환(Recurrent)

4 RAG가 하지 않는 일은? 이름 _____

① 관련 문서를 찾는다 ② 찾은 문서를 프롬프트에 넣는다 ③ LLM의 가중치를 다시 학습시킨다

5 RAG를 우리말로 옮기면 검색 _____ 생성이다. 이름 _____

6 이 자료의 실습에서 LLM으로 쓴 모델의 파라미터는 약 _____억 개다. 이름 _____

7 LLM에게 그냥 묻는 것과 RAG로 묻는 것의 차이를 오픈북 시험에 빗대어 설명해 보세요. 이름 _____

원문: 24-01 랭체인을 시작하기 전에 — https://wikidocs.net/287136

24-02 RAG 이해하기

핵심 한 줄

문서를 미리 벡터로 바꿔 두고, 질문도 벡터로 바꿔 코사인 유사도가 높은 문서 몇 개를 찾아 질문과 함께 LLM에 넘기는 것이 RAG의 뼈대다.

왜 배우나

RAG의 품질은 "알맞은 문서를 찾아오는가"에서 대부분 정해집니다. 찾아오는 방법이 임베딩과 코사인 유사도이고, 11장과 19-05, 20-06에서 본 것을 그대로 씁니다. 이 절에서 가장 작은 검색기를 직접 만들어 보면 뒤의 벡터 데이터베이스가 무엇을 대신해 주는지 알 수 있습니다.

1) 텍스트 임베딩과 코사인 유사도

정의

- 벡터(vector): 숫자를 순서대로 늘어놓은 것. 예: $[1,\ 0.2,\ 0.5,\ 7]$은 원소가 4개인 벡터.
- 임베딩(embedding): 글을 벡터로 바꾸는 것. 단어를 바꾸면 워드 임베딩(12장), 문장이나 문서를 벡터 하나로 바꾸면 문장 임베딩·문서 임베딩(19-05).
- 코사인 유사도(cosine similarity)(11장): 두 벡터가 이루는 각의 코사인. -1에서 1 사이이고, 방향이 같을수록 1에 가깝습니다.

$$\cos(\theta) = \frac{A \cdot B}{\|A\|\,\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\,\sqrt{\sum_{i=1}^{n} B_i^2}}$$

기호	뜻
$A,\ B$	비교할 두 벡터 (원소 n개)
$A \cdot B$	내적: 같은 자리의 원소끼리 곱해 모두 더한 값
$\|A\|$	벡터의 길이: 원소를 제곱해 더한 뒤 제곱근

길이로 나누기 때문에 벡터의 크기는 영향을 주지 않고 방향만 봅니다.

예시 적용

숫자로. $A = [0, 1, 1, 1]$, $B = [1, 0, 1, 1]$, $C = [2, 0, 2, 2]$일 때:

$$\cos(A, B) = \frac{0 + 0 + 1 + 1}{\sqrt{3}\,\sqrt{3}} = \frac{2}{3} \approx 0.6667, \qquad \cos(B, C) = \frac{2 + 0 + 2 + 2}{\sqrt{3}\,\sqrt{12}} = \frac{6}{6} = 1$$

C는 B를 두 배 한 것이라 방향이 같습니다. 그래서 유사도가 1입니다.

문장으로(실습 1). 문장 6개를 벡터(768차원)로 바꿔 두고, 질문 "아무것도 안 먹었더니 꼬르륵 소리가 나네"와의 유사도를 구했습니다.

문서	질문과의 유사도
굶어서 허기가 지네요	0.533
저는 배가 고파요	0.489
허기 워기라는 게임이 있는데 즐거워	0.144

질문과 겹치는 단어가 하나도 없는데도 배고픔에 관한 두 문장이 위로 올라왔습니다. "허기"라는 글자가 들어 있는 게임 문장은 3등이지만 점수가 뚝 떨어집니다. "저기 배가 지나가네요"(타는 배)는 3등 안에 들지 못했습니다.

2) RAG 챗봇의 구조

정의

- RAG(Retrieval-Augmented Generation, 검색 증강 생성): 질문으로 관련 문서를 검색하고, 검색 결과를 프롬프트에 넣어 LLM이 그것을 바탕으로 답하게 하는 기술.
- 프롬프트(prompt): LLM에 넣는 글 전체. 지시, 참고할 문서, 질문이 모두 들어갑니다.

언제	하는 일
미리 한 번	참고할 문서를 모두 임베딩해 벡터로 갖고 있는다
질문이 올 때마다	① 질문을 임베딩한다
	② 문서 벡터들과 유사도를 구해 상위 k개를 고른다
	③ 고른 문서와 질문을 프롬프트에 넣어 LLM에 보낸다. 답을 정리하는 것은 LLM의 몫이다

예시 적용

서울시 정책에 답하는 챗봇이라면(교재의 예): 정책 문서들을 미리 벡터로 바꿔 둡니다. "신혼부부를 위한 서울 정책이 궁금해"가 들어오면 이 문장의 벡터와 가장 가까운 정책 문서 3개를 골라, 질문과 함께 LLM에 넘깁니다.

실습 1의 `return_answer_candidate`가 ①과 ②입니다. 여기에 ③만 붙이면 RAG이고, 실습 6에서 붙입니다. 실습 1은 문서 6개를 판다스 표에 넣어 하나씩 비교했지만, 문서가 많아지면 20-07에서 본 벡터 색인이 필요합니다(24-06).

비유

오픈북 시험

외운 것만으로 답하면 모르는 문제는 지어내게 됩니다. 오픈북 시험에서는 문제를 보고 책에서 관련 쪽을 찾아 펼친 다음, 그 쪽을 보고 답을 씁니다. 답이 좋으려면 맞는 쪽을 펼쳐야 합니다.

외운 것만으로 답하기 = = LLM에게 그냥 묻기
관련 쪽 찾아 펼치기 = = 검색 (임베딩 + 유사도)
펼친 쪽을 보며 답 쓰기 = = 검색 결과를 넣은 프롬프트로 생성
책 = = 벡터 데이터베이스에 넣어 둔 문서

실습 1

임베딩과 코사인 유사도만으로 만드는 가장 작은 검색기

```python
# 임베딩과 코사인 유사도만으로 만드는 가장 작은 검색기
import numpy as np
import pandas as pd
from numpy import dot
from numpy.linalg import norm
from sentence_transformers import SentenceTransformer

# 1) 코사인 유사도: 두 벡터가 같은 방향을 볼수록 1에 가깝다
def cos_sim(A, B):
    return dot(A, B) / (norm(A) * norm(B))

vec1 = np.array([0, 1, 1, 1])
vec2 = np.array([1, 0, 1, 1])
vec3 = np.array([2, 0, 2, 2])
print("벡터1과 벡터2의 유사도:", round(cos_sim(vec1, vec2), 4))
print("벡터1과 벡터3의 유사도:", round(cos_sim(vec1, vec3), 4))
print("벡터2와 벡터3의 유사도:", round(cos_sim(vec2, vec3), 4))

# 2) 문장을 벡터로: 교재의 OpenAI 임베딩 대신 내 컴퓨터에서 도는 한국어 SBERT
model = SentenceTransformer("jhgan/ko-sroberta-multitask")

def get_embedding(text):
```

```
    return model.encode(text)

vector = get_embedding("저는 배가 고파요")
print("\n벡터의 모양:", vector.shape, " 앞 5개:", vector[:5].round(3))

# 3) 문서 6개를 미리 벡터로 바꿔 표에 넣어 둔다
data = ["저는 배가 고파요",
        "저기 배가 지나가네요",
        "굶어서 허기가 지네요",
        "허기 워기라는 게임이 있는데 즐거워",
        "스팀에서 재밌는 거 해야지",
        "스팀에어프라이어로 연어구이 해먹을거야"]
df = pd.DataFrame(data, columns=["text"])
df["embedding"] = df["text"].apply(get_embedding)

# 4) 검색: 질문도 벡터로 바꾸고, 모든 문서와 유사도를 구해 높은 순으로 3개
def return_answer_candidate(df, query):
    query_embedding = get_embedding(query)
    df["similarity"] = df["embedding"].apply(lambda x: cos_sim(x, query_embedding))
    return df.sort_values("similarity", ascending=False).head(3)

for query in ["아무것도 안 먹었더니 꼬르륵 소리가 나네", "주말에 할 게임 추천해줘"]:
    print(f"\n질문: {query}")
    for _, row in return_answer_candidate(df, query).iterrows():
        print(f"  {row.similarity:.3f}  {row.text}")
```

실행 결과

```
벡터1과 벡터2의 유사도: 0.6667
벡터1과 벡터3의 유사도: 0.6667
벡터2와 벡터3의 유사도: 1.0

벡터의 모양: (768,)  앞 5개: [ 0.184 -0.507 -0.229  0.043  0.22 ]

질문: 아무것도 안 먹었더니 꼬르륵 소리가 나네
  0.533  굶어서 허기가 지네요
  0.489  저는 배가 고파요
  0.144  허기 워기라는 게임이 있는데 즐거워

질문: 주말에 할 게임 추천해줘
  0.433  허기 워기라는 게임이 있는데 즐거워
  0.413  스팀에서 재밌는 거 해야지
  0.196  스팀에어프라이어로 연어구이 해먹을거야
```

직접 해 보기 `data`에 문장을 몇 개 더 넣고 질문을 바꿔 보세요. "배"가 들어간 문장(먹는 배, 타는 배, 신체의 배)을 섞어 넣으면 뜻에 맞게 찾아 주나요?

24-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 코사인 유사도는 벡터의 길이가 길수록 커진다. (O / X) 이름 _____

2 RAG에서 문서의 임베딩은 질문이 올 때마다 새로 계산한다. (O / X) 이름 _____

3 $[1, 0]$과 $[3, 0]$의 코사인 유사도는? 이름 _____

① 0 ② 1 ③ 3

4 RAG에서 LLM에 넘기는 것은? 이름 _____

① 질문만 ② 검색된 문서만 ③ 검색된 문서와 질문

5 글을 벡터로 바꾸는 것을 _____이라고 한다. 이름 _____

6 실습에서 "아무것도 안 먹었더니 꼬르륵 소리가 나네"와 가장 비슷한 문장의 유사도는 _____이었다. 이름 _____

7 실습의 질문과 1등 문장은 겹치는 단어가 없습니다. 키워드 검색이었다면 어떻게 됐을지, 20-07의 예와 함께 설명해 보세요. 이름 _____

원문: 24-02 RAG(Retrieval-Augmented Generation) 이해하기 — https://wikidocs.net/287141 (교재는 OpenAI의 text-embedding-ada-002로 임베딩합니다. 여기서는 키 없이 도는 한국어 SBERT로 바꿨습니다. 문장 6개와 질문은 교재와 같습니다. 두 번째 질문은 추가한 것입니다)

24-03 랭체인의 두 가지 청킹 전략

핵심 한 줄

긴 문서는 검색의 단위가 될 작은 조각(청크)으로 잘라야 하고, 자르는 방법에는 정해진 길이로 자르는 것과 뜻이 바뀌는 곳에서 자르는 것이 있다.

왜 배우나

책 한 권을 벡터 하나로 바꾸면 무엇에 관한 책인지만 남고 세부 내용은 뭉개집니다. LLM에 넣을 수 있는 글의 길이에도 한계가 있습니다(21-01). 그래서 문서를 잘라서 넣는데, 어디서 자르느냐에 따라 검색이 찾아오는 것이 달라집니다.

1) 길이 단위 청킹

정의

- 청크(chunk): 긴 글을 자른 조각 하나. 임베딩과 검색의 단위가 됩니다. 자르는 일을 청킹(chunking)이라고 합니다.
- RecursiveCharacterTextSplitter: 랭체인에서 가장 널리 쓰는 분할기. 청크가 `chunk_size` 글자를 넘지 않도록, 구분자 `["\n\n", "\n", " ", ""]`를 차례로 써서 자릅니다. 먼저 빈 줄(문단)로 나누고, 그래도 길면 줄바꿈으로, 그래도 길면 공백으로 나눕니다.
- chunk_overlap: 이웃한 청크가 겹치는 글자 수. 앞 청크의 끝부분이 다음 청크의 앞에 다시 들어갑니다. 문장이 경계에서 잘려 뜻이 끊기는 것을 줄입니다.

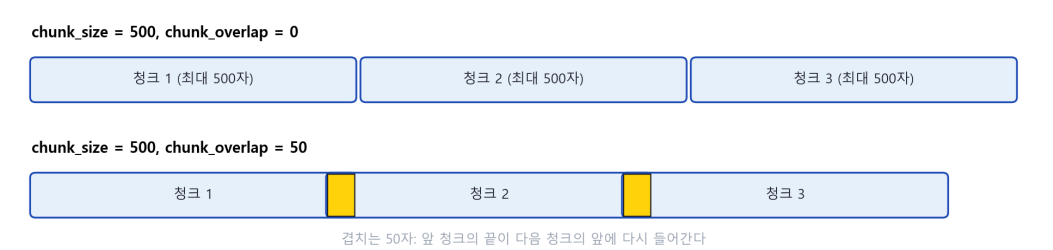


그림 2 겹침이 없을 때와 있을 때

예시 적용

길이 18,085,369자의 뉴스 모음을 잘랐습니다(실습 2).

설정	청크 수	1번 청크의 끝 / 2번 청크의 앞
chunk_size 500, overlap 0	47,068	…강북구 번동 길 위에서 / 사람들이 싸우고 있다 총소리가 났다…
chunk_size 500, overlap 50	48,670	…강북구 번동 길 위에서 / 망치로 이씨 머리를 때렸다 이 과정에서 오후 6시 20분께 강북구 번동…

겹침이 없으면 "번동 길 위에서 / 사람들이 싸우고 있다"처럼 한 문장이 두 청크로 갈라집니다. 뜻을 보고 자른 것이 아니라 글자 수를 세어 잘랐기 때문입니다. 겹침을 주면 2번 청크가 앞 청크의 끝부분부터 시작해, 갈라진 문장이 한쪽에는 온전히 들어갑니다. 대신 청크가 1,602개 늘었습니다.

2) 의미 고려 청킹

정의

- 의미 고려 청킹(semantic chunking): 문장마다 임베딩을 구하고, 이웃한 문장의 뜻이 크게 달라지는 곳에서 자르는 방법. 랭체인의 `SemanticChunker` 가 이 일을 합니다.
- 코사인 거리(cosine distance): $1-$ 코사인 유사도. 0에 가까울수록 비슷하고, 클수록 다릅니다(범위 0~2).

문장이 $s_1, \ldots, s_m$이고 그 벡터가 $v_1, \ldots, v_m$일 때:

$$d_i = 1 - \cos(v_i,\ v_{i+1}) \quad (i = 1, \ldots, m-1), \qquad d_i > \tau \text{ 인 곳에서 자른다}$$

기준 τ를 정하는 방식	기준값
백분위수 (기본값)	거리들을 크기순으로 세웠을 때 아래에서 95%에 해당하는 값. 거리가 상위 5%인 곳에서 자른다
표준편차	평균 + 표준편차의 몇 배 (기본 3배)
사분위수	3사분위수 + 사분위 범위의 몇 배 (기본 1.5배)

예시 적용

교재의 예시 글(7,460자, "박민호"라는 인물의 일대기)을 문장 174개로 나눠 이웃 문장의 거리를 구했습니다(실습 2).

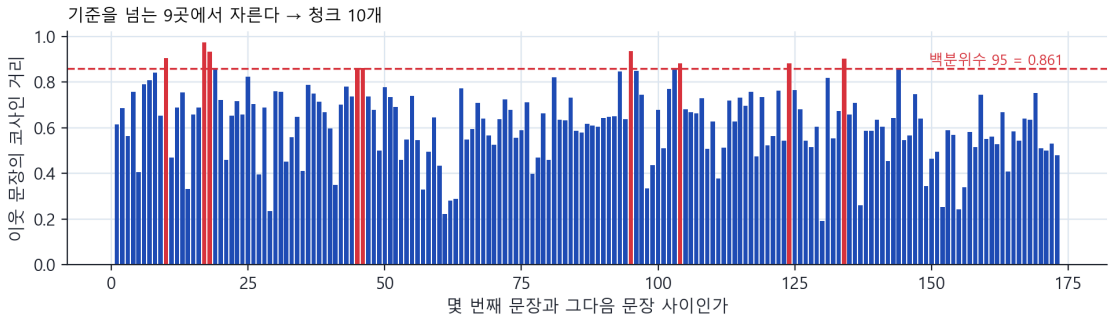


그림 3 이웃한 문장 사이의 코사인 거리 173개. 빨간 막대가 기준(백분위수 95)을 넘은 곳

백분위수	기준 거리	청크 수	청크의 길이
95	0.861	10	477, 309, 33, 1108, 34, 1771, 387, 799, 438, 2095
80	0.747	36	191, 97, 60, 58, 67, …

- 백분위수 95에서 청크가 10개 나왔습니다. 교재가 OpenAI 임베딩으로 얻은 개수와 같습니다.
- 잘린 자리 하나: "…박민호 역시 서울대학교 컴퓨터공학과에 입학했다." / "작은 어촌마을에서 자란 그에게 서울은 전혀 다른 세상이었다." 입시 이야기에서 서울 생활 이야기로 넘어가는 곳입니다.
- 길이가 33자에서 2,095자까지 들쭉날쭉합니다. 길이 단위 청킹과 달리 크기를 보장하지 않습니다. 문장 하나짜리 청크(33자, 34자)도 생깁니다.
- 기준을 낮추면(백분위수 80) 더 잘게 잘립니다.

비유

책을 나눠 철하기

두꺼운 책을 50쪽마다 자르면 크기는 고르지만 문단 한가운데가 잘립니다. 장이 바뀌는 곳에서 자르면 내용은 온전하지만 묶음마다 두께가 다릅니다.

50쪽마다 자르기	= = 길이 단위 청킹
앞 묶음의 마지막 쪽을 한 번 더 넣기	= = chunk_overlap
장이 바뀌는 곳에서 자르기	= = 의미 고려 청킹

실습 2

길이로 자르기, 겹쳐 자르기, 뜻이 바뀌는 곳에서 자르기

```
# 코랩에서는 먼저:  !pip install langchain-text-splitters sentence-transformers
import re
import urllib.request

import numpy as np
from langchain_text_splitters import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer

# 1) 길이 단위 청킹: 아주 긴 뉴스 모음을 500자 이하로 자른다
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/lovit/soynlp/master/tutorials/"
    "2016-10-20.txt", filename="2016-10-20.txt")
with open("2016-10-20.txt", encoding="utf-8") as f:
    file = f.read()
print("텍스트의 길이:", len(file))

text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=0)
texts = text_splitter.create_documents([file])
print("분할된 청크의 수:", len(texts))
print("앞 5개 청크의 길이:", [len(t.page_content) for t in texts[:5]])
print("1번 청크의 끝:", repr(texts[1].page_content[-40:]))
print("2번 청크의 앞:", repr(texts[2].page_content[:40]))

# 2) chunk_overlap: 이웃한 청크가 50자까지 겹치게 한다
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
texts = text_splitter.create_documents([file])
print("\n겹치게 했을 때 청크의 수:", len(texts))
print("1번 청크의 끝:", repr(texts[1].page_content[-40:]))
print("2번 청크의 앞:", repr(texts[2].page_content[:40]))

# 3) 의미 고려 청킹: 이웃한 문장의 뜻이 크게 달라지는 곳에서 자른다
#    (교재의 SemanticChunker가 하는 일을, 로컬 임베딩으로 직접 구현)
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/chatgpt-kr/openai-api-tutorial/main/"
    "ch06/test.txt", filename="test.txt")
with open("test.txt", encoding="utf-8") as f:
    story = f.read()
sentences = [s.strip() for s in re.split(r"(?<=[.?!])\s+", story) if s.strip()]
print("\n텍스트의 길이:", len(story), " 문장 수:", len(sentences))

model = SentenceTransformer("jhgan/ko-sroberta-multitask")
vectors = model.encode(sentences, normalize_embeddings=True)
distances = 1 - (vectors[:-1] * vectors[1:]).sum(axis=1)   # 이웃 문장의 코사인 거리
print("이웃 문장 사이 거리: 평균", distances.mean().round(3),
      " 최대", distances.max().round(3))

def semantic_chunks(percentile):
    threshold = np.percentile(distances, percentile)       # 이 값보다 멀면 자른다
    cuts = [i + 1 for i, d in enumerate(distances) if d > threshold]
    bounds = [0] + cuts + [len(sentences)]
    return [" ".join(sentences[a:b]) for a, b in zip(bounds, bounds[1:])], threshold

for percentile in (95, 80):
    chunks, threshold = semantic_chunks(percentile)
    print(f"백분위수 {percentile} (기준 거리 {threshold:.3f}):",
          f"청크 {len(chunks)}개,",
          "길이", [len(c) for c in chunks][:12])

chunks, _ = semantic_chunks(95)
for i in (1, 2):
    print(f"\n[{i}번 청크의 끝] ...{chunks[i][-70:]}")
    print(f"[{i + 1}번 청크의 앞] {chunks[i + 1][:70]}...")
```

```
실행 결과
텍스트의 길이: 18085369
분할된 청크의 수: 47068
앞 5개 청크의 길이: [17, 498, 496, 497, 202]
1번 청크의 끝: ' 머리를 때렸다  이 과정에서 오후 6시 20분께 강북구 번동 길 위에서'
2번 청크의 앞: '사람들이 싸우고 있다 총소리가 났다 는 등의 신고가 여러건 들어왔다  5'

겹치게 했을 때 청크의 수: 48670
1번 청크의 끝: ' 머리를 때렸다  이 과정에서 오후 6시 20분께 강북구 번동 길 위에서'
2번 청크의 앞: '망치로 이씨 머리를 때렸다  이 과정에서 오후 6시 20분께 강북구 번동'

텍스트의 길이: 7460  문장 수: 174
이웃 문장 사이 거리: 평균 0.619  최대 0.975
백분위수 95 (기준 거리 0.861): 청크 10개, 길이 [477, 309, 33, 1108, 34, 1771, 387, 799, 438, 2095]
백분위수 80 (기준 거리 0.747): 청크 36개, 길이 [191, 97, 60, 58, 67, 165, 143, 33, 35, 209, 267,
23]

[1번 청크의 끝] ...들은 곧바로 대한민국 최고의 대학에 진학할 수 있었고, 이는 엄청난 특혜였다. 박민호 역시
서울대학교 컴퓨터공학과에 입학했다.
[2번 청크의 앞] 작은 어촌마을에서 자란 그에게 서울은 전혀 다른 세상이었다....

[2번 청크의 끝] ...작은 어촌마을에서 자란 그에게 서울은 전혀 다른 세상이었다.
[3번 청크의 앞] 높은 빌딩들과 복잡한 지하철, 그리고 끊임없이 움직이는 사람들. 모든 것이 새롭고 놀라웠다.
서울대학교에서의 4년은 박민호에게 ...
```

직접 해 보기 `chunk_size`를 100과 2000으로 바꿔 청크의 수를 보세요. `semantic_chunks`의 백분위수를 99와 50으로 바꾸면 청크가 몇 개가 되나요? 가장 짧은 청크는 어떤 문장인가요?

확인 문제

24-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 RecursiveCharacterTextSplitter는 문장의 뜻을 파악해 문맥 단위로 자른다. (O / X) 이름 _____

2 의미 고려 청킹으로 자른 청크는 길이가 모두 비슷하다. (O / X) 이름 _____

3 RecursiveCharacterTextSplitter가 가장 먼저 쓰는 구분자는? 이름 _____
① 공백 ② 줄바꿈 한 번 ③ 줄바꿈 두 번(빈 줄)

4 코사인 유사도가 0.3인 두 문장의 코사인 거리는? 이름 _____
① 0.3 ② 0.7 ③ 1.3

5 이웃한 청크가 겹치는 글자 수를 정하는 인자는 chunk_____이다. 이름 _____

6 실습에서 백분위수 95로 자른 청크는 _____개였다. 이름 _____

7 길이 단위 청킹과 의미 고려 청킹의 장단점을 하나씩 말해 보세요. 이름 _____

원문: 24-03 랭체인의 두 가지 청킹 전략(Chunking Strategy) — https://wikidocs.net/287137 (교재는 OpenAI 임베딩을 쓰는 SemanticChunker를 호출합니다. 여기서는 SemanticChunker가 하는 일(문장 임베딩 → 이웃 문장의 거리 → 기준을 넘는 곳에서 자르기)을 로컬 임베딩으로 직접 구현했습니다. 문장을 나누는 규칙과 세부 처리가 달라 잘리는 자리는 교재와 다를 수 있습니다. 표준편차와 사분위수 방식은 정의만 실었습니다)

24-04 ChatOpenAI와 Memory

핵심 한 줄

랭체인에서 LLM은 프롬프트를 받아 답을 돌려주는 부품이고, 프롬프트 템플릿과 | 로 이어 체인을 만든다. LLM은 지난 대화를 기억하지 못하므로, 기억은 지난 대화를 프롬프트에 다시 넣어 주는 것으로 만든다.

왜 배우나

RAG의 마지막 단계는 "지시 + 검색 결과 + 질문"을 한 덩어리의 글로 조립해 LLM에 넣는 일입니다. 그 조립 도구가 프롬프트 템플릿입니다. 또 챗봇이라면 "아까 말한 그거"를 알아들어야 합니다. LLM에는 기억이 없으니, 기억처럼 보이게 만드는 방법을 알아야 합니다.

1) LLM 객체, 프롬프트 템플릿, 체인

정의

- LLM 객체: 프롬프트를 넣으면 답을 돌려주는 부품. 교재에서는 `ChatOpenAI(model_name="gpt-4o")` 로 만들고 `llm.invoke(질문)` 으로 부릅니다.
- 프롬프트 템플릿(prompt template): 빈칸(`{변수}`)이 있는 글. 빈칸에 값을 채우면 프롬프트가 완성됩니다. 같은 지시문을 값만 바꿔 되풀이해 쓸 수 있습니다.
- 체인(chain): 부품을 `|` 로 이어, 앞 부품의 출력이 뒤 부품의 입력으로 흘러가게 한 것. `prompt | llm` 은 "빈칸을 채운 다음 LLM에 넣어라"입니다.
- invoke: 랭체인의 부품을 실행하는 공통 함수. LLM도, 체인도, 뒤에 나올 리트리버도 모두 `invoke` 로 부릅니다.

예시 적용

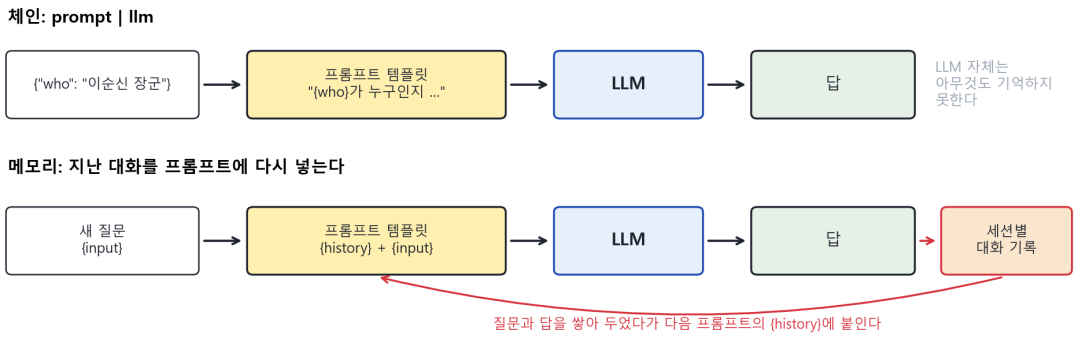


그림 4 체인(위)과 메모리(아래)

직접 호출(실습 3-2). "세종대왕이 누구인지 한 문장으로 설명해주세요."를 LLM에 그대로 넣었습니다.

모델의 답
세종대왕은 조선의 2대 왕으로, 1408년부터 1450년까지 세자로 지배한 인물입니다.

틀린 답입니다. 세종은 조선의 4대 왕이고 재위 기간은 1418년부터 1450년입니다. 문장은 매끄럽지만 내용은 지어냈습니다. 21-02에서 본 대로 모델은 그럴듯한 다음 토큰을 고를 뿐이고, 15억 개짜리 작은 모델은 아는 것도 적습니다. 이것이 할루시네이션이고, 이 장에서 RAG를 배우는 이유입니다. (훨씬 큰 모델은 이런 유명한 사실은 대체로 맞히지만, 학습한 글에 없는 내 문서에 대해서는 똑같은 일이 생깁니다.)

템플릿과 체인. 템플릿 "{who}가 누구인지 한 문장으로 설명해주세요."의 빈칸에 "이순신 장군"을 채우면 "이순신 장군가 누구인지 한 문장으로 설명해주세요."가 됩니다(조사 "가"는 템플릿에 적힌 그대로입니다. 템플릿은 글자를 끼워 넣을 뿐입니다). `llm_chain = prompt | llm` 으로 이은 뒤 `llm_chain.invoke({"who": "이순신 장군"})` 을 부르면, 채워진 글이 LLM으로 넘어가 답이 나옵니다. 이 답에도 "명나라와의 전쟁에서"라는 틀린 내용이 들어 있습니다.

2) 메모리: 지난 대화를 프롬프트에 다시 넣기

정의

- 메모리(memory): 지난 대화를 저장해 두었다가 다음 프롬프트에 붙여 주는 장치. LLM은 호출과 호출 사이에 아무것도 기억하지 못합니다. 매번 프롬프트에 적힌 것만 봅니다.
- 세션(session): 한 사용자가 대화를 시작해 끝낼 때까지의 연속된 대화 하나. 세션마다 `session_id` 를 붙여 대화 기록을 따로 관리합니다. 여러 사람이 동시에 써도 대화가 섞이지 않습니다.
- ChatMessageHistory: 한 세션의 대화 기록(사람의 말, AI의 말)을 차례로 담는 그릇.
- RunnableWithMessageHistory: 체인을 감싸서, 부를 때마다 ① 그 세션의 기록을 꺼내 템플릿의 `{history}` 자리에 넣고 ② 새 질문과 답을 기록에 덧붙이는 일을 대신 해 주는 부품.

대화가 길어지면 프롬프트도 길어집니다. 모델이 한 번에 읽을 수 있는 토큰 수(21-01)를 넘으면 오래된 대화를 버리거나 요약해야 합니다.

예시 적용

세션 A와 세션 B에 같은 질문 "제 이름이 뭐라고 했죠?"를 했습니다(실습 3-2).

세션	앞서 한 말	"제 이름이 뭐라고 했죠?"의 답
A	"제 이름은 세진이고, 요즘 자연어 처리를 공부하고 있어요."	세진이라고 했습니다.
B	없음	죄송합니다, 저는 인공지능으로서 이름을 알고 있는 기능이 없습니다.

왜 A만 아는가. 세션 A의 두 번째 질문 때 모델에 실제로 들어간 프롬프트를 출력해 보면, `Current Conversation:` 아래에 첫 번째 질문과 답이 통째로 들어가 있습니다. 모델이 기억한 것이 아니라, 프롬프트에 적혀 있어서 읽은 것입니다. 세션 B의 프롬프트에는 그 자리가 비어 있습니다.

대화가 끝난 뒤 쌓인 메시지는 세션 A가 4개(질문 2 + 답 2), 세션 B가 2개입니다.

기록은 `[HumanMessage(content='…'), AIMessage(content='…')]` 처럼 파이썬 객체의 모양 그대로 들어갔습니다. 교재의 템플릿을 그대로 쓴 결과이고, 이렇게 넣어도 모델은 알아듣습니다. 실제 서비스에서는 "Human: … / AI: …"처럼 깔끔한 글로 바꿔 넣는 것이 보통입니다.

비유

기억을 못 하는 상담원과 상담 기록지

상담원은 전화를 끊으면 방금 한 이야기를 모두 잊습니다. 그래서 고객마다 기록지를 두고, 전화를 받을 때마다 기록지를 처음부터 다시 읽은 뒤 답합니다. 통화가 끝나면 오늘 한 이야기를 기록지에 덧붙입니다.

기억을 못 하는 상담원 = = LLM
고객별 기록지 = = 세션별 대화 기록
기록지를 다시 읽기 = = 프롬프트의 {history}에 넣기
고객 번호 = = session_id

실습 3 local_llm.py: 교재의 ChatOpenAI 자리에 쓸 로컬 LLM

```python
# 교재의 ChatOpenAI(gpt-4o) 대신 쓰는, 내 컴퓨터에서 도는 작은 LLM (API 키 불필요)
from transformers import AutoModelForCausalLM, AutoTokenizer

NAME = "Qwen/Qwen2.5-1.5B-Instruct"                # 15억 파라미터, 약 3GB
tokenizer = AutoTokenizer.from_pretrained(NAME)
model = AutoModelForCausalLM.from_pretrained(NAME)

def ask(prompt, max_new_tokens=120):
    """프롬프트(문자열)를 넣으면 모델의 답(문자열)을 돌려준다."""
    messages = [{"role": "user", "content": prompt}]
    enc = tokenizer.apply_chat_template(messages, add_generation_prompt=True,
                                        return_tensors="pt", return_dict=True)
    out = model.generate(**enc, max_new_tokens=max_new_tokens, do_sample=False)
    new_tokens = out[0, enc["input_ids"].shape[1]:]  # 프롬프트 뒤에 새로 쓴 부분만
    return tokenizer.decode(new_tokens, skip_special_tokens=True).strip()

if __name__ == "__main__":
    print("파라미터 수:", f"{sum(p.numel() for p in model.parameters()):,}")
    print(ask("RAG가 무엇인지 한 문장으로 설명해줘.", max_new_tokens=60))
```

실행 결과

```
파라미터 수: 1,543,714,304
RAG는 Retrieval-Augmented Generation의 약자로, 기존의 GPT 모델을 훈련시켜서 특정 질문에 대한 답변을 제공하는 데 사용되는 방법입니다.
```

직접 해 보기 OpenAI API 키가 있다면 이 파일 대신 교재처럼 `llm = ChatOpenAI(model_name="gpt-4o")`를 쓰면 됩니다. 뒤의 실습에서 `ask(글)`을 `llm.invoke(글).content`로 바꾸면 그대로 돌아갑니다. 출력에 나온 RAG의 설명("기존의 GPT 모델을 훈련시켜서…")은 틀렸습니다. RAG는 모델을 훈련시키지 않습니다. 15억 개짜리 작은 모델의 한계이고, 아래 실습들의 답도 이 점을 감안해 읽어야 합니다.

실습 3-2 프롬프트 템플릿, 체인, 메모리

```python
# 코랩에서는 먼저:  !pip install langchain-core langchain-community transformers
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.messages import AIMessage
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnableLambda
from langchain_core.runnables.history import RunnableWithMessageHistory

from local_llm import ask

# 1) LLM 객체: 프롬프트를 받아 답을 돌려주는 부품. 교재의 ChatOpenAI 자리다
sent = []                                          # 모델에 실제로 들어간 프롬프트

def call_llm(prompt):
    text = prompt if isinstance(prompt, str) else prompt.to_string()
    sent.append(text)
    return AIMessage(content=ask(text, max_new_tokens=60))

llm = RunnableLambda(call_llm)
question = "세종대왕이 누구인지 한 문장으로 설명해주세요."
print("[직접 호출]", llm.invoke(question).content)

# 2) 프롬프트 템플릿: 빈칸이 있는 문장. 빈칸만 바꿔 가며 다시 쓴다
prompt = PromptTemplate.from_template("{who}가 누구인지 한 문장으로 설명해주세요.")
print("\n[템플릿을 채운 결과]", prompt.format(who="이순신 장군"))

# 3) 체인: 템플릿의 출력이 LLM의 입력으로 흘러간다 ( | 로 잇는다)
llm_chain = prompt | llm
print("[체인 호출]", llm_chain.invoke({"who": "이순신 장군"}).content)

# 4) 메모리: 지난 대화를 프롬프트의 {history} 자리에 다시 넣어 준다
template = """아래는 사람과 AI의 친근한 대화입니다. AI의 이름은 위키독스봇입니다.
대화 문맥을 바탕으로 한 문장으로 답하세요.

Current Conversation:
{history}

Human: {input}
AI:"""
chain = PromptTemplate.from_template(template) | llm
store = {}                                         # 세션 id → 그 세션의 대화 기록
```

```python
def get_history(session_id):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

with_message_history = RunnableWithMessageHistory(
    chain, get_history, input_messages_key="input", history_messages_key="history")

def chat(session_id, text):
    result = with_message_history.invoke(
        {"input": text}, config={"configurable": {"session_id": session_id}})
    print(f"  ({session_id}) 사람: {text}\n  ({session_id}) AI  : {result.content}")

print("\n[메모리]")
chat("A", "제 이름은 세진이고, 요즘 자연어 처리를 공부하고 있어요.")
chat("A", "제 이름이 뭐라고 했죠?")
chat("B", "제 이름이 뭐라고 했죠?")                  # 다른 세션: 기록이 없다

print("\n세션별로 쌓인 메시지 수:", {k: len(v.messages) for k, v in store.items()})
print("\n[세션 A의 두 번째 질문 때 모델에 들어간 프롬프트]")
print(sent[-2])
```

실행 결과

```
[직접 호출] 세종대왕은 조선의 2대 왕으로, 1408년부터 1450년까지 세자로 지배한 인물입니다.

[템플릿을 채운 결과] 이순신 장군가 누구인지 한 문장으로 설명해주세요.
[체인 호출] 이순신 장군은 조선시대의 전사로, 명나라와의 전쟁에서 뛰어난 전략과 투병력을 발휘하여 여러 대승을 이끌었으며, 한국 역사상 가장 유명한 전사 중 하나입니다

[메모리]
  (A) 사람: 제 이름은 세진이고, 요즘 자연어 처리를 공부하고 있어요.
  (A) AI  : AI: 네, 좋은 취미입니다! 어떻게 도와드릴까요?
  (A) 사람: 제 이름이 뭐라고 했죠?
  (A) AI  : 세진이라고 했습니다.
  (B) 사람: 제 이름이 뭐라고 했죠?
  (B) AI  : 죄송합니다, 저는 인공지능으로서 이름을 알고 있는 기능이 없습니다.

세션별로 쌓인 메시지 수: {'A': 4, 'B': 2}

[세션 A의 두 번째 질문 때 모델에 들어간 프롬프트]
아래는 사람과 AI의 친근한 대화입니다. AI의 이름은 위키독스봇입니다.
대화 문맥을 바탕으로 한 문장으로 답하세요.

Current Conversation:
[HumanMessage(content='제 이름은 세진이고, 요즘 자연어 처리를 공부하고 있어요.', additional_kwargs=
{}, response_metadata={}), AIMessage(content='AI: 네, 좋은 취미입니다! 어떻게 도와드릴까요?',
additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[])]

Human: 제 이름이 뭐라고 했죠?
AI:
```

직접 해 보기 세션 A에 질문을 두세 개 더 이어 보세요. 맨 처음에 말한 이름을 계속 기억하나요? `sent[-1]` 을 출력해 프롬프트가 얼마나 길어졌는지 보세요.

24-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 LLM은 같은 사용자의 지난 호출 내용을 스스로 기억하고 있다. (O / X) 이름 _____

2 세션이 다르면 대화 기록도 따로 관리된다. (O / X) 이름 _____

3 `prompt | llm` 에서 | 가 하는 일은? 이름 _____

① 둘 중 하나를 고른다 ② 앞의 출력을 뒤의 입력으로 넘긴다 ③ 두 결과를 더한다

4 대화가 길어질수록 커지는 것은? 이름 _____

① 모델의 파라미터 수 ② 프롬프트의 길이 ③ 단어 집합의 크기

5 빈칸이 있는 글에 값을 채워 프롬프트를 만드는 도구를 프롬프트 _____이라고 한다. 이름 _____

6 랭체인의 부품을 실행하는 공통 함수의 이름은 _____이다. 이름 _____

7 실습의 세션 B가 이름을 알지 못한 이유를, 모델에 들어간 프롬프트를 근거로 설명해 보세요. 이름 _____

원문: 24-04 ChatOpenAI와 Memory — https://wikidocs.net/287154 (교재는 ChatOpenAI로 gpt-4o를 부릅니다. 여기서는 API 키 없이 돌도록, 로컬 LLM을 RunnableLambda로 감싸 같은 자리에 끼웠습니다. 프롬프트 템플릿, 체인, RunnableWithMessageHistory를 쓰는 법은 교재와 같습니다. 질문은 작은 모델에 맞게 짧은 것으로 바꿨고, 세션을 둘로 나눠 비교하는 부분과 실제 프롬프트를 출력하는 부분은 추가한 것입니다)

24-05 랭체인의 다양한 PDF 로더

핵심 한 줄

PDF 로더는 PDF를 읽어 본문(page_content)과 출처 정보(metadata)를 가진 Document의 목록으로 바꿔 주고, 로더마다 속도와 뽑아내는 글, 메타데이터가 다르다.

왜 배우나

RAG에 넣고 싶은 문서는 대부분 PDF입니다. PDF는 글자의 위치만 적어 둔 형식이라, 글을 뽑아내는 도구에 따라 줄바꿈과 순서가 달라집니다. 잘못 뽑힌 글은 뒤의 청킹, 임베딩, 검색, 답변까지 모두 망칩니다. RAG의 첫 단추입니다.

1) Document와 세 가지 로더

정의

- 문서 로더(document loader): 파일이나 웹페이지를 읽어 랭체인의 Document 목록으로 바꿔 주는 부품.
- Document: 랭체인이 글 조각을 담는 그릇. 두 가지가 들어 있습니다. `page_content` 는 본문 글, `metadata` 는 그 글의 출처 정보(파일 이름 `source` , 쪽 번호 `page` 등)입니다.

`로더(파일명).load()` 는 쪽마다 Document 하나를 만듭니다. `load_and_split()` 은 거기에 분할기까지 적용합니다.

로더	속도	metadata	쓰임
PyPDFLoader	보통	기본적인 것 (source, page 등)	가장 널리 쓴다
PyMuPDFLoader	빠르다	많다 (제목, 저자, 전체 쪽수, 만든 프로그램 등)	파일이 많거나 길 때
PDFPlumberLoader	느리다	많다	뽑힌 글의 순서가 다른 로더와 다를 때 비교용

예시 적용

통일부의 「2023 북한인권보고서」(448쪽)를 세 로더로 읽었습니다(실습 4).

로더	걸린 시간	Document 수 (글자가 있는 것)	metadata 항목 수
PyPDFLoader	27.1초	448 (445)	9
PyMuPDFLoader	3.4초	448 (445)	16
PDFPlumberLoader	36.8초	448 (445)	9

- PyMuPDFLoader가 PyPDFLoader보다 약 8배, PDFPlumberLoader보다 약 11배 빨랐습니다. 순서는 교재의 결과(15.5초, 2.4초, 30.5초)와 같습니다. 시간 자체는 다른 작업이 함께 돌던 컴퓨터에서 잰 것이라 참고만 하세요.
- 448쪽 중 3쪽은 글자가 없는 쪽(그림, 백지)입니다. 교재의 "청크의 수 445"가 이 숫자입니다.
- **같은 쪽인데 뽑힌 글이 다릅니다.** PyPDFLoader와 PyMuPDFLoader는 "2023 북한인권보고서\n06\n간 지연되고…"처럼 쪽 머리말이 본문 앞에 붙고, PDFPlumberLoader는 본문 "간 지연되고…"부터 시작합니다(교재의 출력을 보면 이 로더는 머리말을 쪽의 끝에 붙입니다). 머리말 같은 군더더기가 청크마다 들어가면 검색을 흐리므로, 실제로는 뽑힌 글을 눈으로 확인하고 정리해야 합니다(24-07의 전처리).

실습 4 같은 PDF를 세 가지 로더로 읽어 비교하기

```
# 코랩에서는 먼저:  !pip install langchain-community pypdf pymupdf pdfplumber
import time
import urllib.request

from langchain_community.document_loaders import PDFPlumberLoader
from langchain_community.document_loaders import PyMuPDFLoader
from langchain_community.document_loaders import PyPDFLoader

FILE = "2023_북한인권보고서.pdf"
urllib.request.urlretrieve(
    "https://github.com/chatgpt-kr/openai-api-tutorial/raw/main/ch06/"
    "2023_%EB%B6%81%ED%95%9C%EC%9D%B8%EA%B6%8C%EB%B3%B4%EA%B3%A0%EC%84%9C.pdf",
    filename=FILE)

# 같은 PDF를 세 가지 로더로 읽어 걸린 시간과 결과를 비교한다
for Loader in (PyPDFLoader, PyMuPDFLoader, PDFPlumberLoader):
    start = time.time()
    pages = Loader(FILE).load()                      # 페이지 하나가 Document 하나
    took = time.time() - start
    filled = [p for p in pages if p.page_content.strip()]
    doc = filled[5]
    print(f"\n[{Loader.__name__}] {took:.1f}초, 문서 {len(pages)}개",
          f"(글자가 있는 것 {len(filled)}개)")
    print("  metadata의 키:", sorted(doc.metadata)[:8],
          f"... 모두 {len(doc.metadata)}개")
    print("  본문 앞부분  :", repr(doc.page_content[:60]))
```

실행 결과

```
[PyPDFLoader] 27.1초, 문서 448개 (글자가 있는 것 445개)
  metadata의 키: ['creationdate', 'creator', 'moddate', 'page', 'page_label', 'producer',
'source', 'total_pages'] ... 모두 9개
  본문 앞부분  : '2023 북한인권보고서\n06\n간 지연되고 있는 북한인권재단이 출범하고, 북한인권법이 하루 속
\n히 정상적으로 '

[PyMuPDFLoader] 3.4초, 문서 448개 (글자가 있는 것 445개)
  metadata의 키: ['author', 'creationDate', 'creationdate', 'creator', 'file_path', 'format',
'keywords', 'modDate'] ... 모두 16개
  본문 앞부분  : '2023 북한인권보고서\n06\n간 지연되고 있는 북한인권재단이 출범하고, 북한인권법이 하루 속
\n히 정상적으로 '

[PDFPlumberLoader] 36.8초, 문서 448개 (글자가 있는 것 445개)
  metadata의 키: ['CreationDate', 'Creator', 'ModDate', 'Producer', 'Trapped', 'file_path',
'page', 'source'] ... 모두 9개
  본문 앞부분  : '간 지연되고 있는 북한인권재단이 출범하고, 북한인권법이 하루 속\n히 정상적으로 이행되는
토대가 되기를 바랍니'
```

직접 해 보기 가지고 있는 PDF(강의 자료, 논문)로 바꿔 돌려 보세요. 표나 2단 편집이 있는 쪽은 로더마다 글의 순서가 어떻게 달라지나요?

24-05 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 어떤 PDF 로더를 쓰든 뽑혀 나오는 글은 완전히 같다. (O / X) 이름 _____

2 Document의 metadata에는 그 글이 어느 파일의 몇 쪽에서 왔는지가 들어 있다. (O / X) 이름 _____

3 실습에서 가장 빨랐던 로더는? 이름 _____

① PyPDFLoader ② PyMuPDFLoader ③ PDFPlumberLoader

4 Document에서 본문 글이 들어 있는 곳은? 이름 _____

① page_content ② metadata ③ source

5 448쪽짜리 PDF에서 글자가 있는 쪽은 _____쪽이었다. 이름 _____

6 PyMuPDFLoader의 metadata 항목은 _____개였다. 이름 _____

7 RAG 챗봇이 답과 함께 "출처: ○○.pdf 41쪽"을 보여 주려면 무엇이 필요한지 설명해 보세요. 이름 _____

원문: 24-05 랭체인의 다양한 PDF 로더 — https://wikidocs.net/287138 (교재는 주피터의 %%time으로 시간을 재고 load_and_split()을 씁니다. 여기서는 일반 파이썬 파일로도 돌도록 time.time()으로 쟀고, 쪽 단위 결과를 보려고 load()를 썼습니다)

24-06 벡터 데이터베이스

핵심 한 줄

벡터 데이터베이스는 청크의 글, 벡터, 메타데이터를 함께 넣어 두고 질문과 가까운 청크를 찾아 주는 저장소로, 임베딩과 검색을 함수 한두 개로 대신해 준다.

왜 배우나

24-02에서는 문서 6개를 판다스 표에 넣고 유사도를 하나씩 계산했습니다. 문서가 수천, 수만 개가 되면 저장, 검색 속도, 파일로 남기기를 모두 직접 챙겨야 합니다. 20-07에서 써 본 Faiss 같은 도구가 이 일을 대신하고, 랭체인은 여러 벡터 데이터베이스를 같은 문법으로 쓰게 해 줍니다.

1) 크로마와 파이스

정의

- 벡터 데이터베이스(vector database): 벡터를 저장하고, 주어진 벡터와 가까운 벡터를 빨리 찾아 주는 데이터베이스. 벡터와 함께 원래 글과 메타데이터도 넣어 둡니다.
- 크로마(Chroma): 설치가 쉽고 폴더 하나에 저장되는 오픈소스 벡터 데이터베이스.
- 파이스(FAISS): 20-07의 벡터 검색 라이브러리. 랭체인이 글과 메타데이터를 함께 관리하도록 감싸 놓았습니다.

하는 일	크로마	파이스
청크를 임베딩해 넣기	`Chroma.from_documents(청크, 임베딩)`	`FAISS.from_documents(청크, 임베딩)`
가까운 청크 찾기	`db.similarity_search(질문, k=4)`	
점수와 함께 찾기	`db.similarity_search_with_score(질문, k=4)`	
파일로 저장	`persist_directory="폴더"` 를 주고 만든다	`db.save_local("폴더")`
파일에서 불러오기	`Chroma(persist_directory=…, embedding_function=…)`	`FAISS.load_local("폴더", 임베딩, …)`

`similarity_search(질문)` 한 줄이 24-02의 ①(질문을 임베딩)과 ②(유사도가 높은 것 고르기)를 모두 합니다. 그래서 데이터베이스를 만들 때 임베딩 모델을 함께 넘겨 둡니다.

임베딩 모델을 끼우는 자리. 랭체인은 `embed_documents` (문서 여러 개 → 벡터 여러 개)와 `embed_query` (질문 하나 → 벡터 하나) 두 함수만 있으면 어떤 임베딩 모델이든 받습니다. 교재는 `OpenAIEmbeddings()` 를, 이 실습은 두 함수를 가진 작은 클래스 `LocalEmbeddings` 를 넣었습니다.

예시 적용

24-05의 「2023 북한인권보고서」 448쪽을 500자 이하로 자르니 청크가 **925개** 나왔습니다(실습 5).

단계	걸린 시간
청크 925개를 임베딩해 크로마에 넣기	468초
저장해 둔 폴더에서 크로마 다시 불러오기	0.0초

시간의 거의 전부가 임베딩입니다(다른 작업이 함께 돌던 CPU 기준). 한 번 해 두고 파일로 남기면 다음부터는 바로 씁니다. 파이스에는 크로마에 든 벡터를 꺼내 그대로 넣어 임베딩을 다시 하지 않았습니다.

검색 결과. "북한의 교육과정"으로 찾은 4개는 350쪽(정치사상 교육에 새 과목 신설), 392쪽(교과과정의 생산노동), 29쪽(사상교육), 283쪽(여성의 사회활동)입니다. 앞의 셋은 교육에 관한 대목이고 넷째는 빗나갔습니다.

머리말이 검색을 흐린다. 파이스에서 "종교 활동을 하면 어떤 처벌을 받나"로 찾은 청크는 모두 "9. 사상·양심 및 종교의 자유에 대한 권리 183 IV. 경제적·사회적·문화적 권리 I. 발간개요…"처럼 **쪽 머리말**로 시작합니다. 2, 3등은 종교의 자유를 다루는 장의 쪽이라 방향은 맞지만, 1등은 다른 장의 쪽입니다. 24-05에서 본 대로 PDF에서 뽑은 글에는 쪽마다 머리말이 섞여 있고, 그것이 청크마다 들어가 벡터를 서로 비슷하게 만듭니다. 24-07에서 전처리를 먼저 하는 이유입니다.

거리. 파이스가 돌려준 점수 0.919는 유사도가 아니라 거리입니다. 작을수록 가깝습니다. 길이가 1인 벡터에서 이 거리는 $2 - 2\cos$이므로 코사인 유사도로는 약 0.54입니다.

실습 5 PDF를 청크로 잘라 크로마와 파이스에 넣고 검색하기

```
# 코랩에서는 먼저:
#   !pip install langchain-community langchain-chroma faiss-cpu pypdf
import os
import shutil
import time
import urllib.request

from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import FAISS
from langchain_core.embeddings import Embeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer

FILE = "2023_북한인권보고서.pdf"
if not os.path.exists(FILE):
    urllib.request.urlretrieve(
        "https://github.com/chatgpt-kr/openai-api-tutorial/raw/main/ch06/"
        "2023_%EB%B6%81%ED%95%9C%EC%9D%B8%EA%B6%8C%EB%B3%B4%EA%B3%A0%EC%84%9C.pdf",
        filename=FILE)

# 1) 임베딩 모델: 교재의 OpenAIEmbeddings 자리에 끼울 로컬 모델
class LocalEmbeddings(Embeddings):
    def __init__(self, name="jhgan/ko-sroberta-multitask"):
        self.model = SentenceTransformer(name)
        self.model.max_seq_length = 512            # 기본값은 128토큰

    def embed_documents(self, texts):                # 문서 여러 개 → 벡터 여러 개
        return self.model.encode(texts, normalize_embeddings=True).tolist()

    def embed_query(self, text):                     # 질문 하나 → 벡터 하나
        return self.embed_documents([text])[0]

embeddings = LocalEmbeddings()
print("임베딩 모델이 한 번에 읽는 최대 토큰 수:", embeddings.model.max_seq_length)

# 2) PDF → 페이지 → 500자 이하의 청크
docs = PyPDFLoader(FILE).load()
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=0)
splitted_docs = text_splitter.split_documents(docs)
print("페이지 수:", len(docs), " 청크 수:", len(splitted_docs))

# 3) 크로마: 청크를 임베딩하면서 데이터베이스에 넣는다. 폴더를 주면 파일로도 남는다
DB_DIR = "./chroma_test.db"
shutil.rmtree(DB_DIR, ignore_errors=True)        # 다시 실행할 때 겹쳐 쌓이지 않게
start = time.time()
db = Chroma.from_documents(splitted_docs, embeddings, collection_name="report",
                           persist_directory=DB_DIR)
took = time.time() - start
print(f"크로마에 넣은 벡터: {db._collection.count()}개 ({took:.0f}초)")

def show(doc, score=None):
    text = " ".join(doc.page_content.split())
    label = f"[{doc.metadata['page']}쪽]" if score is None else f"거리 {score:.3f}"
    print(" ", label, text[:60])
```

```
question = "북한의 교육과정"
print("질문:", question)
for doc in db.similarity_search(question, k=4):       # 가까운 청크 4개
    show(doc)

# 4) 저장해 둔 파일에서 다시 불러오기: 임베딩을 다시 하지 않는다
start = time.time()
db_from_file = Chroma(persist_directory=DB_DIR, embedding_function=embeddings,
                      collection_name="report")
print(f"\n파일에서 불러온 크로마의 벡터 수: {db_from_file._collection.count()}",
      f"({time.time() - start:.1f}초)")

# 5) 파이스: 쓰는 법은 같다. 여기서는 이미 구한 벡터를 꺼내 그대로 넣는다
stored = db.get(include=["embeddings", "documents", "metadatas"])
pairs = list(zip(stored["documents"], stored["embeddings"]))
faiss_db = FAISS.from_embeddings(pairs, embeddings, metadatas=stored["metadatas"])
faiss_db.save_local("faiss_index")
new_db = FAISS.load_local("faiss_index", embeddings,
                          allow_dangerous_deserialization=True)
print("파일에서 불러온 파이스의 벡터 수:", new_db.index.ntotal)
question = "종교 활동을 하면 어떤 처벌을 받나"
print("질문:", question)
for doc, score in new_db.similarity_search_with_score(question, k=3):
    show(doc, score)
```

실행 결과

```
임베딩 모델이 한 번에 읽는 최대 토큰 수: 512
페이지 수: 448  청크 수: 925
크로마에 넣은 벡터: 925개 (468초)
질문: 북한의 교육과정
  [350쪽] 언이 있었다. 기존의 김일성, 김정일, 김정숙 중심의 정치사상 교육 에 김정은 관련 과목이 신설되어 북한의
  [392쪽] 경험한다고 언급하며, 교과과정에 있는 생산노동 이외에 아동 노동 은 금지된다고 답변하였다. 379 수집된 진
  [29쪽] 적 지침으로 삼는다’고 규정하고 있어 김일성-김정일주의가 지도사 상임을 밝히고 있다. 사상교육은 유년기부터
  [283쪽] 담고 있어 여성이 결혼 후 출산과 양육 등으로 사회활동(사회적 노동)을 하지 못하는 관습에 가까운 문화 를

파일에서 불러온 크로마의 벡터 수: 925 (0.0초)
파일에서 불러온 파이스의 벡터 수: 925
질문: 종교 활동을 하면 어떤 처벌을 받나
  거리 0.919 1. 시민적·정치적 권리 29 IV. 경제적·사회적·문화적 권리 I. 발간개요V. 취약계층VI. 특별사안 I
  거리 0.945 9. 사상·양심 및 종교의 자유에 대한 권리 183 IV. 경제적·사회적·문화적 권리 I. 발간개요V. 취약
  거리 0.963 9. 사상·양심 및 종교의 자유에 대한 권리 187 IV. 경제적·사회적·문화적 권리 I. 발간개요V. 취약
```

직접 해 보기 `question` 을 바꿔 보세요. `chunk_size` 를 200과 1000으로 바꾸면 청크의 수와 검색 결과가 어떻게 달라지나요?

24-06 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 벡터 데이터베이스에는 벡터만 들어가고 원래 글은 들어가지 않는다. (O / X) 이름 _____

2 저장해 둔 벡터 데이터베이스를 다시 불러올 때는 문서를 다시 임베딩하지 않아도 된다. (O / X) 이름 _____

3 `similarity_search(질문)` 이 내부에서 가장 먼저 하는 일은? 이름 _____

① 질문을 임베딩한다 ② 문서를 다시 자른다 ③ LLM을 부른다

4 랭체인의 임베딩 부품이 갖춰야 하는 두 함수는? 이름 _____

① fit, predict ② embed_documents, embed_query ③ encode, decode

5 크로마를 파일로 남기려면 _____directory 인자를 준다. 이름 _____

6 실습에서 448쪽의 PDF는 500자 기준으로 _____개의 청크가 되었다. 이름 _____

7 실습에서 가장 오래 걸린 단계와 가장 빨랐던 단계를 말하고, 그래서 벡터 데이터베이스를 파일로 저장해 두는 이유를 설명해 보세요. 이름 _____

원문: 24-06 벡터 데이터베이스(Vector Database) — https://wikidocs.net/287150 (교재는 OpenAIEmbeddings로 임베딩하고 청크 크기를 1,000자로 합니다. 여기서는 로컬 임베딩 모델이 한 번에 512토큰까지만 읽으므로 청크를 500자로 줄였습니다. 교재는 파이스에도 from_documents로 다시 임베딩해 넣지만, 여기서는 시간을 아끼려고 크로마에 넣어 둔 벡터를 꺼내 from_embeddings로 넣었습니다)

24-07 RAG를 이용한 한국은행 금융 챗봇

핵심 한 줄

RAG 챗봇은 "문서 읽기 → 전처리 → 벡터 데이터베이스에 넣기"를 미리 해 두고, 질문이 오면 "검색 → 프롬프트에 검색 결과와 질문 채우기 → LLM 호출"을 차례로 하는 함수 하나다.

왜 배우나

지금까지 만든 부품을 하나로 묶습니다. 묶고 나면 RAG가 어떤 질문에 강하고 어디서 틀리는지를 직접 볼 수 있습니다. 25장의 고급 기법은 모두 여기서 본 약점을 고치려는 것입니다.

1) 리트리버와 RAG 프롬프트

정의

- 리트리버(retriever, 검색기): 질문을 받아 관련 Document의 목록을 돌려주는 부품. `vectordb.as_retriever(search_kwargs={"k": 2})` 로 벡터 데이터베이스에서 만들고, `retriever.invoke(질문)` 으로 부릅니다. k는 가져올 청크의 수입니다.
- 컨텍스트(context): 프롬프트에 넣는 검색 결과. 검색된 청크의 `page_content` 를 이어 붙인 글입니다.
- 전처리: 답에 쓸 내용이 아닌 청크(머리말, 목차, 맺음말)를 미리 빼는 일. 넣어 두면 검색에 걸려 엉뚱한 답의 근거가 됩니다.
- temperature: LLM이 다음 토큰을 얼마나 무작위로 고를지 정하는 값(21-02). 0이면 가장 확률이 높은 것만 골라, 같은 질문에 같은 답을 합니다. 근거에 충실해야 하는 RAG에서는 낮게 둡니다.

RAG 프롬프트의 세 부분.

부분	내용	왜 넣나
지시	역할("금융 용어를 설명하는 금융쟁이"), 말투, "검색 결과를 바탕으로 답하라", "검색 결과에 없으면 답할 수 없다고 하라"	LLM이 자기가 아는 것으로 지어내지 않게 한다
`{context}`	검색된 청크	답의 근거
`{question}`	사용자의 질문	

$$\text{답} = \text{LLM}\big(\text{지시} + \underbrace{\text{검색}(q,\ k)}_{\text{컨텍스트}} + q\big)$$

예시 적용

준비. 「경제금융용어 700선」을 `load_and_split()` 으로 읽으니 366개였습니다. 0번은 머리말("찾아보기 한국은행은 국민들이…"), 13번부터가 용어 설명("가계부실위험지수(HDRI)…")입니다. 앞 13개와 맨 끝 1개를 빼고 **352개**를 크로마에 넣었습니다(실습 6). 리트리버에 "비트코인이 궁금해"를 넣으면 155쪽의 비트코인 항목이 1등으로 나옵니다.

질문 넷의 결과.

질문	참고한 쪽	답 (요약)	평가
디커플링이란 무엇인가?	114, 183	"한 나라의 경제가 … 세계 경제의 흐름과 다르게 움직이는 현상" 등	뜻은 맞지만 항목 이름을 "국가 간 경제 동조화"라고 붙였습니다(디커플링은 **탈**동조화). 답이 길어 중간에 끊겼습니다

너는 뭘하는 챗봇이니?	339, 122	"저는 한국은행에서 만든 금융 용어를 설명해주는 금융쟁이입니다…"	프롬프트의 지시를 그대로 따랐습니다
손흥민의 소속 팀은 어디야?	155, 57	"…맨체스터 유나이티드(Fulham) 에所属합니다."	**실패.** 용어집에 없는 내용인데 "답할 수 없다"고 하지 않고 지어냈습니다
HDRI가 100을 넘는 가구는?		검색 없이: "고위험 가구" / 검색 포함: "위험가구 … 고위험가구, 고DTA가구, 고DSR가구로 구분"	검색 없이는 틀렸고, 검색을 붙이니 용어집의 정의대로 맞혔습니다

- **RAG가 해 준 일:** 마지막 줄이 RAG의 효과를 그대로 보여 줍니다. 같은 모델이 검색 결과 없이는 "고위험 가구"라고 틀리게 답했고, 용어집의 해당 쪽을 받자 "위험가구"라고 맞게 답하고 세부 구분까지 덧붙였습니다.
- **RAG가 못 해 준 일:** 손흥민 질문에서는 검색 결과(비트코인 쪽 등)에 답이 없는데도 모델이 지어냈고, 엉뚱한 팀 이름과 중국어 글자까지 섞였습니다. "검색 결과에 없으면 답할 수 없다고 하라"는 지시를 15억 개짜리 작은 모델은 지키지 못했습니다. 큰 모델은 이런 지시를 훨씬 잘 따르고, 작은 모델이라도 이런 상황을 학습시키면 나아집니다(26장).
- 프롬프트가 2,400~2,600자로 깁니다. 쪽 단위 청크 두 개가 통째로 들어가기 때문입니다. 청크를 작게 자르면 프롬프트가 짧아지고 근거도 더 정확해집니다(25장에서는 300자로 자릅니다).

참고

웹 화면 붙이기: 그라디오

교재는 마지막에 그라디오(Gradio)로 채팅 화면을 붙입니다. 답을 돌려주는 함수 하나만 있으면 몇 줄로 웹 화면이 됩니다. 코랩에서 아래를 실습 6 뒤에 이어 실행하면 됩니다(`!pip install gradio`).

```
import gradio as gr

def respond(message, history):
    return query_rag(message)["result"]

gr.ChatInterface(respond, title="금융쟁이").launch()
```

이 자료를 만들 때는 화면을 띄울 수 없어 이 부분은 실행하지 않았습니다.

실습 6

한국은행 「경제금융용어 700선」으로 답하는 RAG 챗봇

```
# 코랩에서는 먼저:  !pip install langchain-community langchain-chroma pypdf
import urllib.request

from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_core.embeddings import Embeddings
from langchain_core.prompts import ChatPromptTemplate
from sentence_transformers import SentenceTransformer

from local_llm import ask

# 1) 문서 읽기: 한국은행 「경제금융용어 700선」
FILE = "2020_경제금융용어 700선_게시.pdf"
urllib.request.urlretrieve(
    "https://github.com/chatgpt-kr/openai-api-tutorial/raw/main/ch07/"
    "2020_%EA%B2%BD%EC%A0%9C%EA%B8%88%EC%9C%B5%EC%9A%A9%EC%96%B4%20700%EC%84%A0_"
    "%EA%B2%8C%EC%8B%9C.pdf", filename=FILE)
texts = PyPDFLoader(FILE).load_and_split()
print("문서의 수:", len(texts))

# 2) 전처리: 머리말과 목차(앞 13개), 맺음말(마지막 1개)은 답에 쓸 내용이 아니다
print("0번 청크 :", repr(texts[0].page_content[:40]))
print("13번 청크:", repr(texts[13].page_content[:40]))
texts = texts[13:-1]
print("남은 청크의 수:", len(texts))

# 3) 임베딩해서 벡터 데이터베이스에 넣기
class LocalEmbeddings(Embeddings):
```

```
    def __init__(self, name="jhgan/ko-sroberta-multitask"):
        self.model = SentenceTransformer(name)
        self.model.max_seq_length = 512

    def embed_documents(self, texts):
        return self.model.encode(texts, normalize_embeddings=True).tolist()

    def embed_query(self, text):
        return self.embed_documents([text])[0]

vectordb = Chroma.from_documents(texts, LocalEmbeddings(), collection_name="bok")
print("벡터 데이터베이스에 든 청크:", vectordb._collection.count())

# 4) 리트리버: 질문과 가장 가까운 청크 2개를 가져오는 검색기
retriever = vectordb.as_retriever(search_kwargs={"k": 2})
for doc in retriever.invoke("비트코인이 궁금해"):
    text = doc.page_content.replace("\n", " ")
    print(f"  [{doc.metadata['page']}쪽] {text[:60]}")

# 5) 프롬프트 템플릿: 검색 결과와 질문이 들어갈 자리를 비워 둔다
prompt = ChatPromptTemplate.from_template("""
당신은 한국은행에서 만든 금융 용어를 설명해주는 금융쟁이입니다.
주어진 검색 결과를 바탕으로 세 문장 이내로 답변하세요.
검색 결과에 없는 내용이라면 답변할 수 없다고 하세요. 반말로 친근하게 답변하세요.

검색 결과:
{context}

질문: {question}

답변:
""")

# 6) 검색 → 프롬프트 채우기 → 생성
def query_rag(question):
    docs = retriever.invoke(question)                              # 검색
    context = "\n\n".join(doc.page_content for doc in docs)        # 검색 결과 합치기
    message = prompt.format(question=question, context=context)  # 빈칸 채우기
    return {"query": question, "result": ask(message, max_new_tokens=150),
            "source_documents": docs, "prompt_length": len(message)}

for question in ["디커플링이란 무엇인가?", "너는 뭘하는 챗봇이니?",
                 "손흥민의 소속 팀은 어디야?"]:
    response = query_rag(question)
    pages = [doc.metadata["page"] for doc in response["source_documents"]]
    print(f"\n질문: {question}  (참고한 쪽: {pages},",
          f"프롬프트 {response['prompt_length']}자)")
    print("답변:", response["result"])

# 7) 검색 없이 LLM에게만 물으면?
print("\n[검색 없이] 가계부실위험지수(HDRI)가 100을 넘는 가구를 뭐라고 불러?")
print("답변:", ask("가계부실위험지수(HDRI)가 100을 넘는 가구를 뭐라고 불러? "
                   "한 문장으로 답해.", max_new_tokens=60))
response = query_rag("가계부실위험지수(HDRI)가 100을 넘는 가구를 뭐라고 불러?")
print("[검색 포함] 답변:", response["result"])
```

실행 결과

```
문서의 수: 366
0번 청크 : 'iii\n찾아보기\n한국은행은 국민들이 경제 및 금융에 대한 이해도를 높이고'
13번 청크: '1\n¬ \n¬\n가계부실위험지수(HDRI)\n가구의 소득 흐름은 물론 금융 및'
남은 청크의 수: 352
벡터 데이터베이스에 든 청크: 352
  [155쪽] 139 ㅂ  비트코인 비트코인(bitcoin)은 가상통화(암호통화)이자 디지털 지급시스템이다. 비트코인 시스
  [152쪽] 136 경제금융용어 700선 간 대화의 깊이와 폭도 더욱 넓어지고 있다. 브릭스는 현재의 경제성장 속도와
앞

질문: 디커플링이란 무엇인가?  (참고한 쪽: [114, 183], 프롬프트 2387자)
답변: 디커플링은 두 가지 의미로 사용됩니다:

1. **국가 간 경제 동조화**: 한 나라의 경제가 다른 국가나 전반적인 세계 경제의 흐름과 다르게 움직이는 현상을
말합니다. 예를 들어, 신흥국가나 유럽 지역 국가가 미국 경제와 다른 모습을 보이는 것처럼 합니다.

2. **종류별 경제 동조화**: 특정 종류의 경제 변수가 다른 종류의 경제 변수와 동일하거나 유사한 흐름을 보이는
현상을 말합니다. 예를 들어, 주가가 하락하면서 통화가 강세를

질문: 너는 뭘하는 챗봇이니?  (참고한 쪽: [339, 122], 프롬프트 2438자)
답변: 저는 한국은행에서 만든 금융 용어를 설명해주는 금융쟁이입니다. 제가 도움이 필요한 금융 용어에 대해
```

```
물어보시는 분들을 돕습니다. 궁금하신 용어에 대해 말씀해 주세요!

질문: 손흥민의 소속 팀은 어디야?  (참고한 쪽: [155, 57], 프롬프트 2588자)
답변: 손흥민 선수가 현재 프리미어리그의 헝거맨팀인 맨체스터 유나이티드(Fulham)에所属합니다.

[검색 없이] 가계부실위험지수(HDRI)가 100을 넘는 가구를 뭐라고 불러?
답변: 가계부 실위험이 100이상인 가구를 '고위험 가구'라고 부릅니다.
[검색 포함] 답변: 가계부실위험지수(HDRI)가 100을 넘는 가구를 '위험가구'라고 불릅니다. 이들은 소득 및 자산
측면에서 모두 취약한 '고위험가구', 자산 측면에서 취약한 '고DTA가구', 소득 측면에서 취약한 '고DSR가구'로
구분할 수 있습니다.
```

직접 해 보기 금융 용어를 골라 물어보세요. `search_kwargs={"k": 2}`를 1과 4로 바꾸면 답이 어떻게 달라지나요? 프롬프트에서 "검색 결과에 없는 내용이라면 답변할 수 없다고 하세요"를 지우면 용어집에 없는 것을 물었을 때 어떻게 답하나요?

확인 문제

24-07 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 RAG 챗봇을 만들려면 LLM을 내 문서로 파인 튜닝해야 한다. (O / X) 이름 _____

2 목차와 머리말 같은 청크는 벡터 데이터베이스에 넣기 전에 빼는 것이 좋다. (O / X) 이름 _____

3 리트리버가 돌려주는 것은? 이름 _____

① 최종 답 ② 질문과 관련된 Document의 목록 ③ 프롬프트 템플릿

4 RAG 프롬프트에 들어가지 않는 것은? 이름 _____

① 검색된 청크 ② 사용자의 질문 ③ 벡터 데이터베이스의 모든 청크

5 프롬프트에 넣는 검색 결과를 _____(context)라고 한다. 이름 _____

6 실습에서 전처리 뒤에 남은 청크는 _____개다. 이름 _____

7 RAG 챗봇의 답이 틀렸을 때, 원인이 검색에 있는지 LLM에 있는지 어떻게 가려낼 수 있을지 설명해 보세요. 이름 _____

원문: 24-07 RAG를 이용한 한국은행 금융 챗봇 — https://wikidocs.net/287159 (교재는 OpenAIEmbeddings와 gpt-4o를 씁니다. 여기서는 로컬 임베딩과 로컬 LLM으로 바꿨고, 작은 모델에 맞게 프롬프트에 "세 문장 이내로"를 넣고 개발자 이름을 뺐습니다. 용어집에 없는 것을 묻는 질문과 검색 없이 묻는 비교는 추가한 것입니다. 그라디오 화면은 코드만 싣고 실행하지 않았습니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
RAG	검색한 문서를 프롬프트에 넣어 LLM이 그것을 보고 답하게 하는 기술	질문 → 청크 2개 → 프롬프트 → 답
임베딩 검색	질문과 문서를 벡터로 바꿔 코사인 유사도로 찾기	"꼬르륵" → "허기가 지네요" 0.533
청킹	문서를 검색 단위의 조각으로 자르기	500자씩 47,068개 / 뜻이 바뀌는 9곳에서 10개
chunk_overlap	이웃 청크가 겹치는 글자 수	50자 겹침 → 48,670개
문서 로더와 Document	파일을 page_content + metadata의 목록으로	448쪽 PDF: 3.4초 ~ 36.8초
벡터 데이터베이스	글, 벡터, 메타데이터를 넣어 두고 가까운 것을 찾는 저장소	Chroma, FAISS
프롬프트 템플릿, 체인	빈칸이 있는 글 / 부품을 | 로 이은 것	prompt | llm
메모리	지난 대화를 프롬프트에 다시 넣어 주는 장치	{history} + {input}
리트리버	질문을 받아 관련 Document를 돌려주는 부품	as_retriever(k=2)

다음

다음 자료: 25장 고급 RAG 기법 (10/16 자료 5/5)

오늘의 챗봇은 질문과 가장 가까운 청크 2개를 그대로 믿었습니다. 그 2개가 틀리면 답도 틀립니다. 다음 자료에서는 찾아온 후보를 다시 채점해 순서를 바꾸는 리랭킹과, 질문 대신 가상의 답으로 검색하는 하이드(HyDE)를 봅니다.

정답과 해설

24-01 랭체인을 시작하기 전에

1. X — 검색, 프롬프트 조립, LLM 호출을 직접 짜도 됩니다.
2. X — LLM을 쓰는 코드를 묶어 주는 도구입니다.
3. ①
4. ③
5. 증강
6. 15
7. 예시 답: 그냥 묻는 것은 외운 것만으로 답하는 시험이라 모르는 문제는 지어내게 된다. RAG는 책을 펴 놓고 보는 오픈북 시험이라, 문제와 관련된 쪽을 찾아 그것을 보고 답한다.

24-02 RAG 이해하기

1. X — 길이로 나누므로 방향만 봅니다.
2. X — 문서는 미리 한 번, 질문만 매번 임베딩합니다.
3. ② — 방향이 같습니다.
4. ③
5. 임베딩
6. 0.533
7. 예시 답: 겹치는 단어가 없어 키워드 검색으로는 찾지 못한다. 20-07에서 gasoline이 든 제목이 0개였는데 시맨틱 검색은 petrol이 든 제목을 찾은 것과 같다.

24-03 랭체인의 두 가지 청킹 전략

1. X — 구분자와 글자 수로 자릅니다.
2. X — 실습에서 33자부터 2,095자까지였습니다.
3. ③
4. ② — 1 − 0.3
5. overlap
6. 10
7. 예시 답: 길이 단위는 빠르고 크기가 고르지만 문장 한가운데가 잘릴 수 있다. 의미 고려는 뜻이 바뀌는 곳에서 잘라 내용이 온전하지만, 문장마다 임베딩이 필요해 느리고(유료 API라면 비용도 든다) 청크의 크기가 들쭉날쭉하다.

24-04 ChatOpenAI와 Memory

1. X — 프롬프트에 적힌 것만 봅니다.
2. O
3. ②
4. ②
5. 템플릿
6. invoke
7. 예시 답: 세션 B의 기록은 비어 있어서, 모델에 들어간 프롬프트의 대화 내용 자리에 이름이 나온 대화가 없었다. LLM은 프롬프트에 없는 것은 알 수 없다.

24-05 랭체인의 다양한 PDF 로더

1. X — 줄바꿈과 머리말의 위치가 달랐습니다.
2. O — source, page

3. ② — 3.4초
4. ①
5. 445
6. 16
7. 예시 답: 청크마다 metadata에 파일 이름(source)과 쪽 번호(page)가 들어 있어야 하고, 검색된 Document의 metadata를 답과 함께 돌려주면 된다.

24-06 벡터 데이터베이스

1. X — 글과 메타데이터도 함께 넣습니다.
2. O
3. ①
4. ②
5. persist
6. 925
7. 예시 답: 청크를 임베딩해 넣는 단계가 가장 오래 걸렸고, 저장한 것을 불러오는 것과 검색은 금방이었다. 임베딩은 한 번만 하고 결과를 파일로 남겨 두면, 다음부터는 그 시간을 다시 쓰지 않아도 된다.

24-07 RAG를 이용한 한국은행 금융 챗봇

1. X — 가중치는 그대로 두고 프롬프트에 문서를 넣습니다.
2. O
3. ②
4. ③
5. 컨텍스트
6. 352
7. 예시 답: 검색된 청크(source_documents)를 직접 읽어 본다. 답의 근거가 그 안에 없으면 검색의 문제이고, 근거가 있는데도 답이 틀렸으면 LLM(또는 프롬프트)의 문제다.

용어 정리

용어	정의
RAG (검색 증강 생성)	질문과 관련된 문서를 검색해 프롬프트에 넣고 LLM이 그것을 바탕으로 답하게 하는 기술
랭체인 (LangChain)	LLM 서비스의 부품을 같은 문법으로 묶어 주는 파이썬 프레임워크
임베딩	글을 벡터로 바꾸는 것
코사인 유사도 / 코사인 거리	두 벡터가 이루는 각의 코사인 / 1에서 코사인 유사도를 뺀 값
프롬프트	LLM에 넣는 글 전체 (지시, 참고 문서, 질문)
청크 / 청킹	긴 글을 자른 조각 / 자르는 일
chunk_size / chunk_overlap	청크의 최대 글자 수 / 이웃 청크가 겹치는 글자 수
의미 고려 청킹	이웃 문장의 뜻이 크게 달라지는 곳에서 자르는 청킹
Document	본문(page_content)과 출처 정보(metadata)를 담는 랭체인의 그릇
문서 로더	파일을 읽어 Document의 목록으로 바꾸는 부품
벡터 데이터베이스	벡터, 글, 메타데이터를 저장하고 가까운 벡터를 찾아 주는 저장소 (Chroma, FAISS)
프롬프트 템플릿	빈칸이 있는 글. 값을 채우면 프롬프트가 된다
체인	부품을 | 로 이어 앞의 출력이 뒤의 입력이 되게 한 것
메모리 / 세션	지난 대화를 프롬프트에 다시 넣어 주는 장치 / 연속된 대화 하나
리트리버	질문을 받아 관련 Document의 목록을 돌려주는 부품
컨텍스트	프롬프트에 넣는 검색 결과